



Cascade

The web-native turbomachinery design environment

Software Capabilities Report

Version 0.1.0 · pre-release

Date 2026-06-01

AGPL-3.0 core · MIT SDK

Cascade is a browser-based turbomachinery design environment for small engineering teams. It carries a design from thermodynamic cycle, through preliminary mean-line aero of the rotating machine, into geometry and rotor dynamics — with no install. Its philosophy is **transparency** (every loss model carries a published citation), **reproducibility** (units-aware, git-diffable TOML projects), and **validation** (a public suite against canonical published cases instead of marketing accuracy claims).

1,115

TESTS PASSING

130

VALIDATION PASS-GATES

<200 ms

CYCLE SOLVE

~30 s

2,000 CANDIDATES

Py 3.12

NUMERICAL CORE

CORE CAPABILITIES (V1)

0D cycle	Brayton variants — simple, recuperated, reheat, intercooled. Real-gas EOS (NASA polynomials + CoolProp; REFPROP optional). Solves η_{thermal} in <200 ms.
Mean-line design	Radial inflow turbine & centrifugal compressor with citable loss models (Whitfield–Baines, Aungier, Wiesner slip).
Exploration	Sobol' sampling of 100s–1000s of candidate geometries; interactive filter-and-pick with live 3D preview.
Performance maps	Full-factorial grids with explicit per-point convergence codes (surge / choke / non-converged).
Rotor dynamics	Timoshenko beam-FEM with gyroscopics: critical-speed map, unbalance response (Bode + amplification + separation margin), Campbell, stability.
Bearings	Plain-journal solver (Reynolds + Christopherson PSOR); tabulated K–C input for any bearing type.
Geometry	Parametric impeller / turbine / volute / rotor; optional STEP & IGES export (OpenCASCADE).
Optimization	SLSQP, NSGA-II, CMA-ES, Powell (BOBYQA API) — single & multi-objective.
Materials & mfg.	Materials database/registry; rule-based manufacturability checks and reports.
Interfaces	Python SDK + <code>cascade</code> CLI at parity with the web UI; sandboxed plugin / adapter system.

ARCHITECTURE & STACK

Core	Python 3.12 · numpy · scipy · Pint (units) · Pydantic · CoolProp
API	FastAPI + Uvicorn · SQLAlchemy 2 (async) + Alembic · Celery + Redis · ReportLab · auto OpenAPI
Frontend	Next.js 15 / React 19 App Router · Plotly · react-three-fiber (WebGL 3D)
Data	PostgreSQL 16 + pgvector
Project format	.cascade directory of units-aware TOML; async collab via git branch + diff + PR
Deploy	Fly.io
Quality	pytest + hypothesis · ruff · mypy (strict)

VALIDATION STATUS

- **CYC-1** simple Brayton: η_{th} 44.80% vs textbook 44.79% (± 0.01 pt) ✓
- **CYC-2** recuperated Brayton: η_{th} 54.91% ✓
- **CYC-3** Capstone C30: η_e 26.09% vs 26% ✓
- **CC-1** Eckardt Rotor A (calibrated): π_{tt} within ± 0.10 ✓
- **RIT-1** NASA TN D-7508: η_{ts} 0.817 vs ~ 0.84 (± 5 pt) ✓
- All implemented pass-gates pass within published tolerances; subset runs in ≈ 14 s.

HONEST SCOPE BOUNDARIES

NOT IN v1 1D thermal-fluid network solver (`cascade.network`) and the **axial turbine / compressor mean-line solver** are specified but not yet built. Several validation cases (CYC-4/5/6/7, RIT-3/4, CC-3/4/5, AXT, TFN) are characterization-only or untested.

v1.1 Real-time co-edit (Yjs/Hocuspocus), multi-stage radial, multi-spool / cooled-turbine matching, tilt-pad / thrust / foil bearings, 2D streamline-curvature throughflow, Bayesian optimization, visual node-graph editor.

ADAPTER-ONLY RANS CFD (OpenFOAM stub) and full 3D FEA (CalculiX adapter; v1 ships 2D-axisymmetric disc stress).

HERO WORKFLOW

Cycle Recuperated Brayton microturbine <200 ms	➤	Design Radial turbine, cycle exit as BC	➤	Explore 2,000 Sobol candidates ~30 s	➤	Map Surge / choke lines ~60 s	➤	Rotor dyn. Bode + crit-speed + margin ~5 s
--	---	---	---	--	---	---	---	--

Sources: README.md, SPEC_SHEET.md, VALIDATION_REPORT.md, KNOWN_GAPS.md, pyproject.toml & source inspection — capabilities reflect actual v1 implementation status, not headline spec claims.